



T34 vs. Tiger.

Lesson 1. Create your own mission.

Author: Spogreev Ivan. Head of Sonar Games.

In this tutorial I will try to tell you in detail how to make your own mission for T34vsTiger. To do this we need to run the game and editor for the game.

Introduction.

The game editor in T34vsTiger is a pretty powerful tool. It allows you to view game objects, models, effects and animation. In it we can create / edit levels and immediately play them, we can run any mission and edit them on the fly.

The entire game is represented in the scripts. Scripts have the extension *. script and have a Java like language. The main scripts are in the folder Scripts, they described the entire game logic and the basic behaviour of objects, etc. The same is true for each mission, having its own scripts characterizing it. They are in their respective folders missions.

For example, the first mission for Russia would be in

"Missions \ Campaign_1 \ Mission_1".

In these scripts are described the arrangement of objects on the level, the conditions of failure and success of the mission and AI behaviour for the units.

So the first thing that we need to do to create our own mission is to create separate folder for it.

Step 1. Separate folder for the mission

Create a new folder for our mission call it Mission1. In principle, it does not matter where you create the folder but for generality, let's create it in the folder Missions. And since we may have a lot of missions, use another shared folder (MyMission) to set up our levels. Thus, you should see a new path:

Mission \ MyMission \ Mission1.

Next you must create all necessary files. We simply copy the contents from the test mission, which you will find at "Missions \ MISSIONS \ TESTMISSION".
Copy all the files into our folder. ~~This will be blank for further changes.~~

Step 2. Edit the scripts of the mission.

In the scripts we describe classes of objects. We can not allow duplicate class names to occur. We now have just such a situation in that we have copied files. We need to modify them and rename the classes.

The list of files that need to be changed are:

Atmosphere.script

Content.script

Mission.script

MissionTasks.script

MissionTestStrings.script

Terrain.script

WorldMatricies.script

As we see, all the files with the extension *. script.

We will make changes in the following order.

First, edit the **atmosphere.script**. This sets the lighting setup, fog, shadows, etc. for the level. If we look at what it described class **TestAtmosphere**. We should rename the class to a name of our own, that is specific for this particular mission.

We will call it Mission1Atmosphere.

<pre>class <u>TestAtmosphere</u> extends CCommonAtmosphere { boolean IsLightEnabled = true; boolean IsAntiSunEnabled = true; Vector SunDirection = new Vector(-0.051952, -0.155542, -0.155348); Color AmbientLight = new Color(47.0 / 255.0, 47.0 / 255.0, 62.0 / 255.0); Color SunColor = new Color(239.0 / 255.0, 236.0 / 255.0, 231.0 / 255.0); Color AntiSunColor = new Color(162.0 / 255.0, 174.0 / 255.0, 185.0 / 255.0); float DistanceToSun = 2000.0;</pre>	<pre>class <u>Mission1Atmosphere</u> extends CCommonAtmosphere { boolean IsLightEnabled = true; boolean IsAntiSunEnabled = true; Vector SunDirection = new Vector(-0.051952, -0.155542, -0.155348); Color AmbientLight = new Color(47.0 / 255.0, 47.0 / 255.0, 62.0 / 255.0); Color SunColor = new Color(239.0 / 255.0, 236.0 / 255.0, 231.0 / 255.0); Color AntiSunColor = new Color(162.0 / 255.0, 174.0 / 255.0, 185.0 / 255.0); float DistanceToSun = 2000.0;</pre>
--	--

Next, make changes to the **Content.script**. This script contains a description of the objects on the level, their position, property, etc. It is generated automatically when you save level.

Rename the class name, file path and name of the class as a string.

<pre> class <u>TestContent</u> { static String m_ObjectListClassFile = "<u>Missions/MISSIONS/TestMission/Content.sc</u> <u>ript</u>"; static String m_ObjectListClassName = "<u>TestContent</u>"; static Array m_MissionObjectList = [["Atmosphere", ..., ..., new Matrix(1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000),],], } </pre>	<pre> class <u>Mission1Content</u> { static String m_ObjectListClassFile = "<u>Missions/MyMission/Mission1/Content.scri</u> <u>pt</u>"; static String m_ObjectListClassName = "<u>Mission1Content</u>"; static Array m_MissionObjectList = [["Atmosphere", ..., ..., new Matrix(1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000, 0.000000, 0.000000, 0.000000, 0.000000, 1.000000),],], } </pre>
--	--

In addition, you must remove all the objects that we do not require that were created to test the mission, which we took as a model. We should be left only with the names of objects:

Atmosphere

MainPlayerUnit

Terrain

CrossRoad2_micro (*it, too, like you can remove, but it will be my Scrin present*)

*The name of the object - this is the first parameter in an object record (understand, when you find appropriate names). Deleting everything else, you should receive the following picture (do not forget to remove the comma after recording **CrossRoad2_micro**):*

** not sure on this one, I left **CrossRoad2_micro** in my folder*

```

class MissionContent
{
    static String m_ObjectListClassFile = "Missions/MyMission/Mission1/Content.script";
    static String m_ObjectListClassName = "Mission1Content";

    static Array m_MissionObjectList = [
        [
            "Atmosphere",
            "",
            "",
            new Matrix(
                1.000000, 0.000000, 0.000000, 0.000000,
                0.000000, 1.000000, 0.000000, 0.000000,
                0.000000, 0.000000, 1.000000, 0.000000,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["FogColorXPos", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogColorXNeg", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogColorYPos", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogColorYNeg", new Color(0.815686, 0.850980, 0.882353, 1.000000)],
                ["FogNear", 10.000000],
                ["FogFar", 800.000000],
                ["FogFarMax", 3500.000000],
                ["EnableHorizontAdjustment", true],
                ["SunColor", new Color(0.937255, 0.925490, 0.905882, 1.000000)],
                ["SunIntensity", 1.000000],
                ["AntiSunColor", new Color(0.811765, 0.819608, 0.854902, 1.000000)],
                ["AntiSunIntensity", 0.300000],
                ["AntiSunAngle", 10.000000],
                ["SunDirection", new Vector(-0.005952, -0.155542, -0.305348)],
                ["AmbientLight", new Color(0.200000, 0.200000, 0.200000, 1.000000)],
                ["ShadowColor", new Color(0.400000, 0.400000, 0.400000, 1.000000)],
                ["FogMode", "Exp"],
                ["FogDensity", 0.000300],
                ["TreeLightKoeF", 0.000000],
                ["TreeNormalRelax", 0.000000]
            ]
        ],
        [
            "MainPlayerUnit",
            "GameObject",
            "CTankT34_85_44Unit",
            new Matrix(
                -0.977255, -0.212067, 0.000000, 3822.968994,
                0.212067, -0.977255, 0.000000, 6137.772949,
                0.000000, 0.000000, 1.000000, 608.984070,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["IsPlayer", true],
                ["IsManual", true],
                ["Affiliation", "FRIEND"],
                ["SurfaceControl", "None"]
            ]
        ],
        [
            "Terrain",
            "",
            "",
            new Matrix(
                1.000000, 0.000000, 0.000000, 0.000000,
                0.000000, 1.000000, 0.000000, 0.000000,
                0.000000, 0.000000, 1.000000, 0.000000,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["ForestTexturesScale", 0.500000],
                ["ForestShadowBordersScale", 0.400000],
                ["ForestShadowAlphaScale", 0.500000],
                ["ForestBeginFadeDist", [250.000000, 480.000000]],
                ["VertForestDistMin", 800.000000],
                ["VertForestDistMax", 8000.000000],
                ["VertForestAngleMin", 0.000000],
                ["VertForestAngleMax", 45.000000],
                ["VertForestBeginFadeDist", [250.000000, 410.000000]],
                ["TerrainShadowPower", 1.000000],
                ["IsAntiLighting", true],
                ["WaterFresnelPower", 2.000000]
            ]
        ],
        [
            "CrossRoad2_micro",
            "Locator",
            "CTerrainPatch",
            new Matrix(
                0.011889, -0.999929, 0.000000, 5185.122070,
                0.999929, 0.011889, 0.000000, 2949.116211,
                0.000000, 0.000000, 1.000000, 586.236694,
                0.000000, 0.000000, 0.000000, 1.000000
            ),
            [
                ["Dimensions", new Vector(15.000000, 15.000000, 0.350586)],
                ["TextureName", "Textures/CrossRoad_micro.tex"],
                ["ZoneName", "Textures/CrossRoad_zone.bmp"],
                ["BorderSize", 2.000000]
            ]
        ]
    ];

    static Array m_MissionCameraScriptList = [
        [
            0.000000,
            "KEY_START",
            [
                ["CinemaID", "Default"],
                []
            ]
        ]
    ];
}

```

The next action will remove all entries in **MissionTasks.script**. This script is responsible for Task of the units. Task - Model the behaviour of AI. Creating their own problem, we can control units, giving them different commands depending on situations. In this mission, we do not need these properties, especially Tasks, so we can safely clean the file.

```
class tmp_AttackTigers extends CBaseUnitGroup
{
    Array KillList = [
        "tmp_GermanTiger1",
        "tmp_GermanTiger2",
        "tmp_GermanT4_1",
        "MainPlayerUnit"
    ];

    void Init()
    {
        CBaseUnitGroup::Init();
        logWarning("TSovietKillGroup void Init");
        SetEnemyReactionType(ERT_AGGRESSIVE);
        ActivateRadar(true);
        ActivateFire(true);
    }
}

class testresst extends CC1M1Gr_BaseInfantry
{
    float m_InfantrySpeed = 2.00;

    void OnPathEndReached()
    {
        CC1M1Gr_BaseInfantry::Brighten(m_StartPosition);
    }
}

// Left Infantry One Group
class testresst2 extends testresst
{
    String m_PointAttack = "SoIdNP2";
}
```

Scripts **MissionTestStrings.script**, **Terrain.script** **WorldMatricies.script** are not particularly significant for us, so I will not describe them now. So show a picture with changes.

Terrain.script

```
class CTestTerrain extends CBaseTerrain, CBaseZoneMap
{
    CTestTerrain()
    {
        if (!CDebugSettings::LoadWater)
            WaterRegions = [];
    }

    void CreateForesRegions()
    {
        Component Materials = new #MaterialManager<CSTBaseForestC1Skin>();

        RegisterForestRegion(
            [ZMC_Forest01],
            Materials,
            "Missions\MISSIONS\TestMission\Forest_Test.tex",
            [0.0, 8.0, 11.0, 13.0, 14.0],
            [500.0, 500.0, 1000.0, 1500.0, 2000.0], // min distance, start fade
            [1500.0, 1500.0, 2000.0, 2500.0, 2500.0] // max distance, hide layer
        );

        RegisterVerticalForest(
            [ZMC_Forest01],
            "textures\Foreststripe.tex",
            [0.8], // number of rings of vertical forest
            17.0f
        );
    }

    CBaseTerrain::CreateForesRegions();

    // terrain resources
    String SkinClass = "CTestTerrainskin";

    // Sea level for compute air density
    float SeaLevel = 450.0;
    float BaseDensity = 0.125;

    // water regions description
    Array WaterRegions = [
        new CWaterRegion(new Vector(23532.10, 29461.19, 1165.0), 1165.0)
    ];

    Array WaterMirrorMasks = [
        [], [CLASSIFICATOR_SHADOW, CLASSIFICATOR_UI, CLASSIFICATOR_TERRAINPATCH]]
    ];

    float TerrainShadowPower = 10.5;
    boolean IsAntilighting = true;
    boolean IsPlanarWater = true;
}

class CTestTerrainskin extends CBaseTerrainskin
{
    CTestTerrainskin()
    {
        SetupTerrainMainMaterial("Missions/MISSIONS/TestMission/Ind_Test.tex");
        AppendMaterials(MissionSpecificMaterials);
    }

    Array MissionSpecificMaterials = [
    ];
}
```

```
class CMission1Terrain extends CBaseTerrain, CBaseZoneMap
{
    CMission1Terrain()
    {
        if (!CDebugSettings::LoadWater)
            WaterRegions = [];
    }

    void CreateForesRegions()
    {
        Component Materials = new #MaterialManager<CSTBaseForestC1Skin>();

        RegisterForestRegion(
            [ZMC_Forest01],
            Materials,
            "Missions\MyMission\Mission1\Forest_Test.tex",
            [0.0, 8.0, 11.0, 13.0, 14.0],
            [500.0, 500.0, 1000.0, 1500.0, 2000.0], // min distance, start fade
            [1500.0, 1500.0, 2000.0, 2500.0, 2500.0] // max distance, hide layer
        );

        RegisterVerticalForest(
            [ZMC_Forest01],
            "textures\Foreststripe.tex",
            [0.8], // number of rings of vertical forest
            17.0f
        );
    }

    CBaseTerrain::CreateForesRegions();

    // terrain resources
    String SkinClass = "CMission1Terrainskin";

    // Sea level for compute air density
    float SeaLevel = 450.0;
    float BaseDensity = 0.125;

    // water regions description
    Array WaterRegions = [
        new CWaterRegion(new Vector(23532.10, 29461.19, 1165.0), 1165.0)
    ];

    Array WaterMirrorMasks = [
        [], [CLASSIFICATOR_SHADOW, CLASSIFICATOR_UI, CLASSIFICATOR_TERRAINPATCH]]
    ];

    float TerrainShadowPower = 10.5;
    boolean IsAntilighting = true;
    boolean IsPlanarWater = true;
}

class CMission1Terrainskin extends CBaseTerrainskin
{
    CMission1Terrainskin()
    {
        SetupTerrainMainMaterial("Missions/MyMission/Mission1/Ind_Test.tex");
        AppendMaterials(MissionSpecificMaterials);
    }

    Array MissionSpecificMaterials = [
    ];
}
```

MissionTestStrings.script

<pre> class <u>CTestMission Strings</u> extends CCommonStrings { final static WString MissionName = getLocalized("MissionTest", "MissionName"); final static WString BriefingText = getLocalized("MissionTest", "BriefingText"); final static WString Objective01 = getLocalized("MissionTest", "Objective01"); final static WString Objective02 = getLocalized("MissionTest", "Objective02"); final static WString Objective03 = getLocalized("MissionTest", "Objective03"); } </pre>	<pre> class <u>CMission1 Strings</u> extends CCommonStrings { final static WString MissionName = getLocalized("MissionTest", "MissionName"); final static WString BriefingText = getLocalized("MissionTest", "BriefingText"); final static WString Objective01 = getLocalized("MissionTest", "Objective01"); final static WString Objective02 = getLocalized("MissionTest", "Objective02"); final static WString Objective03 = getLocalized("MissionTest", "Objective03"); } </pre>
--	---

WorldMatrices.script

<pre> class <u>TestWorldMatrices</u> extends CWorldMatrices { final static float MatrixWidth = 9000.0; final static float MatrixHeight = 9000.0; } class <u>TestLandscapeLayer</u> extends CLandscapeLayer { final static int ImageWidth = 2049; final static int ImageHeight = 2049; final static String ImageFileName = "<u>Missions/MISSIONS/TestMission/hmap.raw</u>"; final static float FloatValueFactor = 0.070000*257.0; } class <u>TestWaterHeights</u> extends CBaseWaterHeightsLayer { final static String ImageFileName = "<u>Missions/MISSIONS/TestMission/hwater.raw</u>"; final static float FloatValueFactor = 0.070000*257.0; } class <u>TestTerrainZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 1024; final static int ImageHeight = 1024; final static String ImageFileName = "<u>Missions/MISSIONS/TestMission/TerrainZone_Test.bmp</u>"; } class <u>TestRouterZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 2048; final static int ImageHeight = 2048; final static String ImageFileName = "<u>Missions/MISSIONS/TestMission/RouterZone_Test.bmp</u>"; } class <u>TestMicroTextures1</u> extends CMicroTexLayer { final static String ImageFileName = "<u>Missions/MISSIONS/TestMission/micro_Test.bmp</u>"; } </pre>	<pre> class <u>Mission1WorldMatrices</u> extends CWorldMatrices { final static float MatrixWidth = 9000.0; final static float MatrixHeight = 9000.0; } class <u>Mission1LandscapeLayer</u> extends CLandscapeLayer { final static int ImageWidth = 2049; final static int ImageHeight = 2049; final static String ImageFileName = "<u>Missions/MyMission/Mission1/hmap.raw</u>"; final static float FloatValueFactor = 0.070000*257.0; } class <u>Mission1WaterHeights</u> extends CBaseWaterHeightsLayer { final static String ImageFileName = "<u>Missions/MyMission/Mission1/hwater.raw</u>"; final static float FloatValueFactor = 0.070000*257.0; } class <u>Mission1TerrainZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 1024; final static int ImageHeight = 1024; final static String ImageFileName = "<u>Missions/MyMission/Mission1/TerrainZone_Test.bmp</u>"; } class <u>Mission1RouterZoneLayer</u> extends CTerrainZoneLayer, CBaseZoneMap { final static int ImageWidth = 2048; final static int ImageHeight = 2048; final static String ImageFileName = "<u>Missions/MyMission/Mission1/RouterZone_Test.bmp</u>"; } class <u>Mission1MicroTextures1</u> extends CMicroTexLayer { final static String ImageFileName = "<u>Missions/MyMission/Mission1/micro_Test.bmp</u>"; } </pre>
---	---

Next, we need to edit the main script of the mission itself, the file **Mission.script**. Here we should change the class name of the mission and supporting classes it calls.

<pre> class CTestMission extends CSPMission { // // Construction and initialization // String n_LocalTime = "14:20:00"; String n_TerrainMapTextureName = "Textures/C1M3_Map.tex"; static String n_MissionBriefingPicMaterial = "C1M3BriefingPic"; Array n_MissionObjectives = []; boolean isDebug = true; Array n_NaupointsForPlayerMap = []; float CockpitMapMinRange = 1000.0; float CockpitMapMaxRange = 4500.0; int CockpitMapZoomSteps = 10; Vector MarksInitPoint = new Vector(1000.0, 1000.0, 0.0); float CockpitMapNavNameMaxRange = 2000.0; Array CockpitMapAccessBox = [new Vector(2742, 1775.0, 0.0), new Vector(7242.0, 6275.0, 0.0)]; final static Array RouterWorkingZones = []; final static Array DefaultMask = [{"ENEMY", "MainMesh"}, {"NEUTRAL"}]; final static Array Mask = [{"ENEMY", "MainMesh"}, {"NEUTRAL", "InvisibleOnRadar"}]; boolean OnTarg = false; boolean TigerAggr = false; boolean MainPlayerStart = false; void CTestMission() { // Construct mission CSPMission("CTestMission", "TestContent"); // Set mission properties SetMissionTerrain(new #ChunkedTerrain<CTestTerrain>()); SetMissionAtmosphere(new #Atmosphere<TestAtmosphere>()); SetMissionSky(new #SkyObject<CSkyModel>()); if (CDebugSettings::LoadForest) SetMissionForest(new CSTBaseForestC1(GetMissionAtmosphere())); if (CDebugSettings::LoadRoads) SetMissionRoadsParms(new CBaseRoadC1()); if (CDebugSettings::LoadGrass) RegisterObject("Grass", new #Grass<CBaseGrassC1>()); SetMissionWorldMatrices(new #WorldMatrices<TestWorldMatrices>(), [[LAYER_TERRAIN_NAME, "TestLandscapeLayer"], [LAYER_TERRAIN_ZONE, "TestTerrainZoneLayer"], [LAYER_ROUTER_ZONE, "TestRouterZoneLayer"], [LAYER_MICROTEXTURE_MAP1, "TestMicroTextures1"], [LAYER_TERRAIN_WATERHEIGHTS, "TestWaterHeights"]]); SetRouterPrecalculatedGraph(new #RouterPrecalculatedGraph<CRouterPrecalculatedGraph>()); SetRouterMap("RouterMap_Layer1", new #RouterMap<CBaseRouterMap>(), 64, RouterWorkingZones); } </pre>	<pre> class CMission1Mission extends CSPMission { // // Construction and initialization // String n_LocalTime = "14:20:00"; String n_TerrainMapTextureName = "Textures/C1M3_Map.tex"; static String n_MissionBriefingPicMaterial = "C1M3BriefingPic"; Array n_MissionObjectives = []; boolean isDebug = true; Array n_NaupointsForPlayerMap = []; float CockpitMapMinRange = 1000.0; float CockpitMapMaxRange = 4500.0; int CockpitMapZoomSteps = 10; Vector MarksInitPoint = new Vector(1000.0, 1000.0, 0.0); float CockpitMapNavNameMaxRange = 2000.0; Array CockpitMapAccessBox = [new Vector(2742, 1775.0, 0.0), new Vector(7242.0, 6275.0, 0.0)]; final static Array RouterWorkingZones = []; final static Array DefaultMask = [{"ENEMY", "MainMesh"}, {"NEUTRAL"}]; final static Array Mask = [{"ENEMY", "MainMesh"}, {"NEUTRAL", "InvisibleOnRadar"}]; boolean OnTarg = false; boolean TigerAggr = false; boolean MainPlayerStart = false; void CMission1Mission() { // Construct mission CSPMission("CMission1Mission", "Mission1Content"); // Set mission properties SetMissionTerrain(new #ChunkedTerrain<CMission1Terrain>()); SetMissionAtmosphere(new #Atmosphere<Mission1Atmosphere>()); SetMissionSky(new #SkyObject<CSkyModel>()); if (CDebugSettings::LoadForest) SetMissionForest(new CSTBaseForestC1(GetMissionAtmosphere())); if (CDebugSettings::LoadRoads) SetMissionRoadsParms(new CBaseRoadC1()); if (CDebugSettings::LoadGrass) RegisterObject("Grass", new #Grass<CBaseGrassC1>()); SetMissionWorldMatrices(new #WorldMatrices<Mission1WorldMatrices>(), [[LAYER_TERRAIN_NAME, "Mission1LandscapeLayer"], [LAYER_TERRAIN_ZONE, "Mission1TerrainZoneLayer"], [LAYER_ROUTER_ZONE, "Mission1RouterZoneLayer"], [LAYER_MICROTEXTURE_MAP1, "Mission1MicroTextures1"], [LAYER_TERRAIN_WATERHEIGHTS, "Mission1WaterHeights"]]); SetRouterPrecalculatedGraph(new #RouterPrecalculatedGraph<CRouterPrecalculatedGraph>()); SetRouterMap("RouterMap_Layer1", new #RouterMap<CBaseRouterMap>(), 64, RouterWorkingZones); } </pre>
--	--

After all the edits we can test our mission.

Step 3. The inclusion of our mission in the mission list.

In order to be able to download, edit and play our level, we need to add it to the list of missions that appear in the editor. This list is in the script **\Scripts\EditorMenuConfig.script**, variable **MissionLoadList**. Find it and edit by appending the following line:

["My Super Mission (PzU)", "CMission1Mission"]

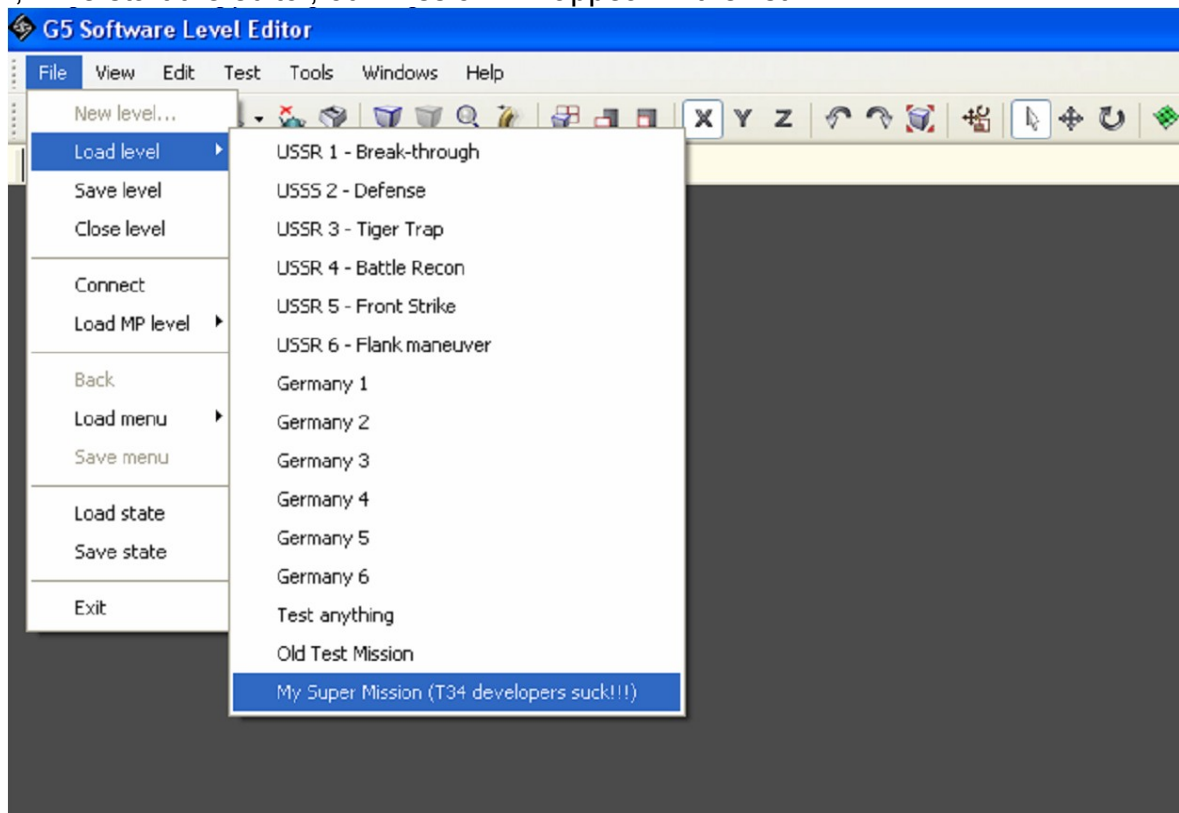
```

class CMenuConfig
{
    CMenuConfig()
    {
        FillZoneEditorTypeList();
    }

    //-----
    // List of missions in editor
    //-----
    final static Array MissionLoadList = [
        ["USSR 1 - Break-through", "CC1M1Mission"],
        ["USSS 2 - Defense", "CC1M2Mission"],
        ["USSR 3 - Tiger Trap", "CC1M3Mission"],
        ["USSR 4 - Battle Recon", "CC1M4Mission"],
        ["USSR 5 - Front Strike", "CC1M5Mission"],
        ["USSR 6 - Flank maneuver", "CC1M6Mission"],
        ["Germany 1", "CC2M1Mission"],
        ["Germany 2", "CC2M2Mission"],
        ["Germany 3", "CC2M3Mission"],
        ["Germany 4", "CC2M4Mission"],
        ["Germany 5", "CC2M5Mission"],
        ["Germany 6", "CC2M6Mission"],
        ["Test anything", "CTestMission"],
        ["Old Test Mission", "CMTMission"],
        ["My Super Mission (T34 developers suck!!!)", "CMission1Mission"]
    ];
}

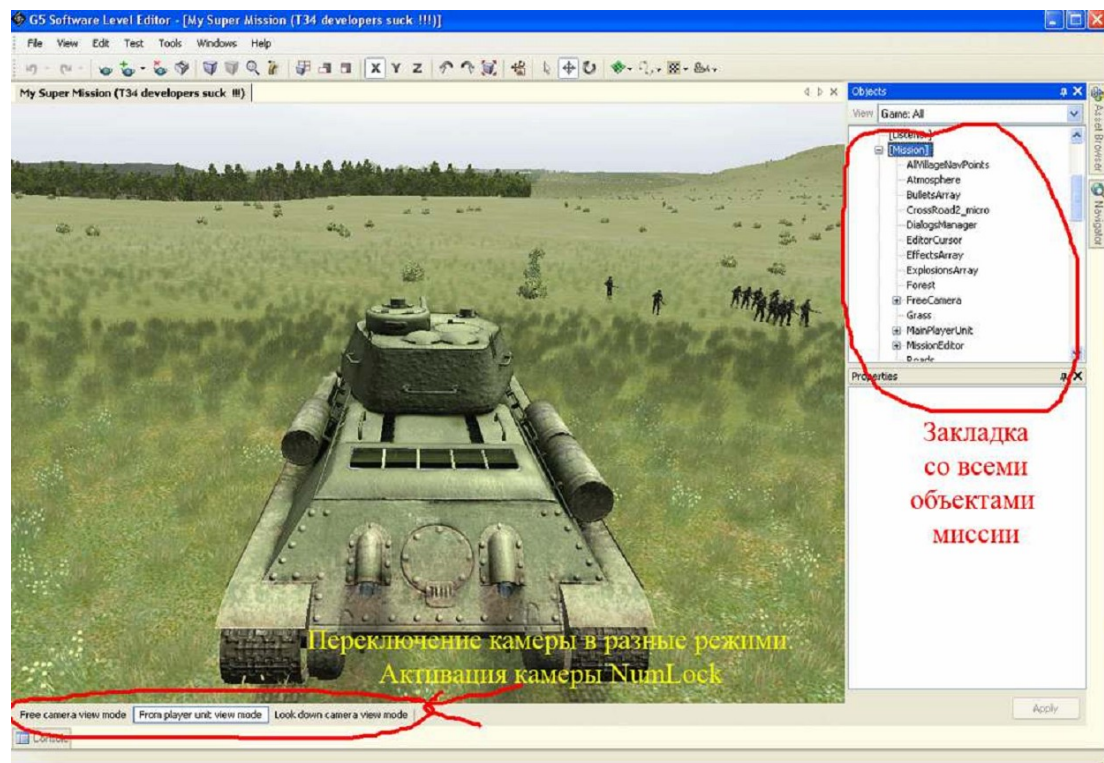
```

Now, if we start the editor, our mission will appear in the list.

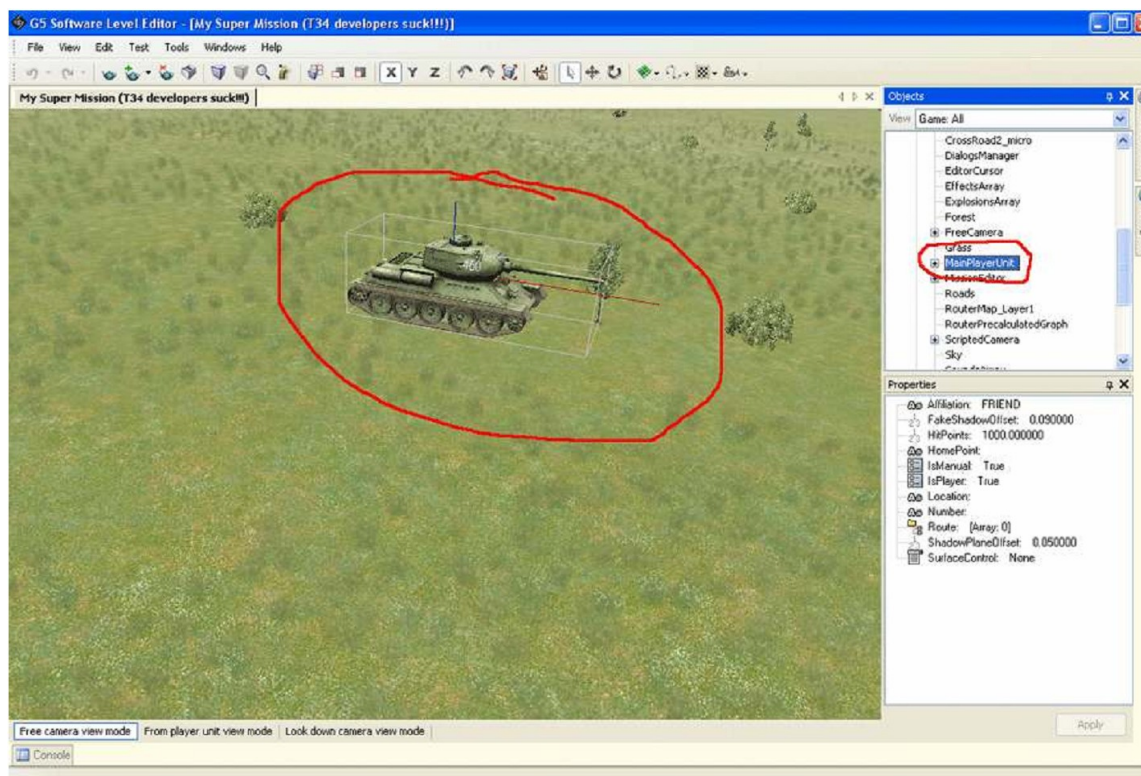


Step 4. Editor Basics.

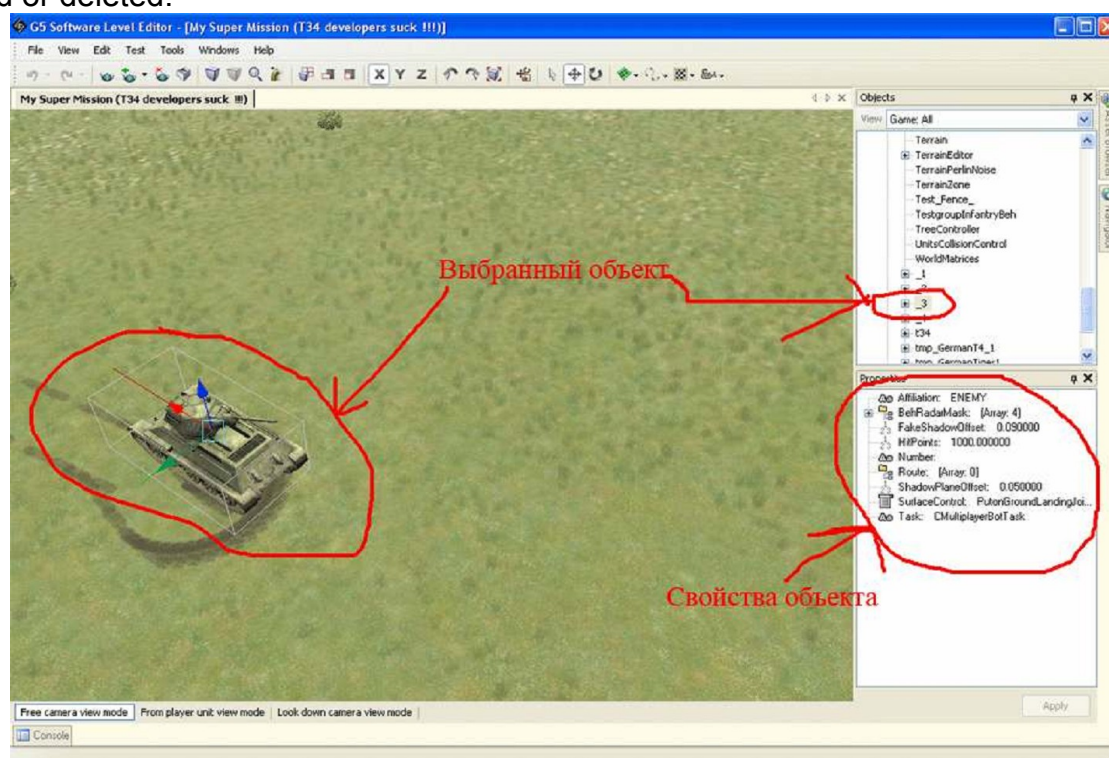
Open the mission and look at the screen. Located on the right is a list of objects in game. We are only interested in playing Mission, as there are all objects of our level.



We can also switch to different camera modes: free, game. When we activate the game camera, then we begin to play immediately. We can choose different objects directly in the right list, or in a free camera to click the right mouse button next to an object (in this case algorithm for choosing the conditions and inaccurate, is not always choose what you want).



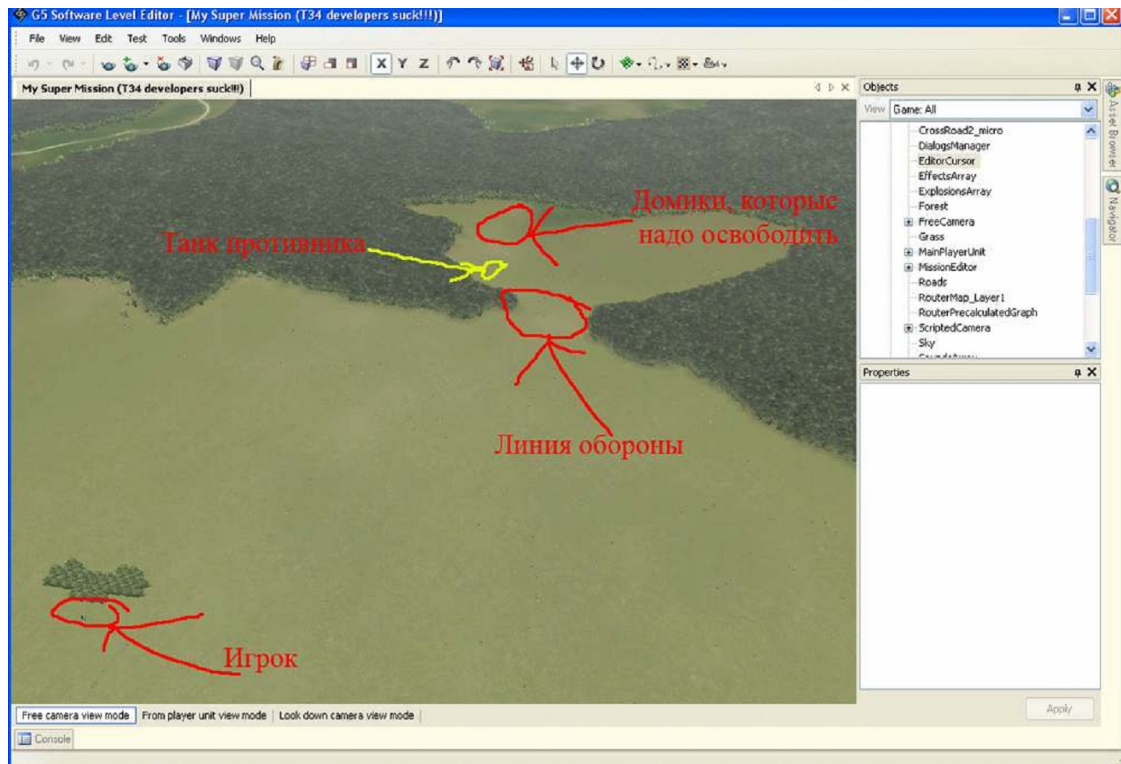
When we choose an object below it shows the properties of an object, they can be edited, added or deleted.



Poklikayte for different objects / properties / icon and see what's what. Note that some times, if you do not click there, the editor may freeze.

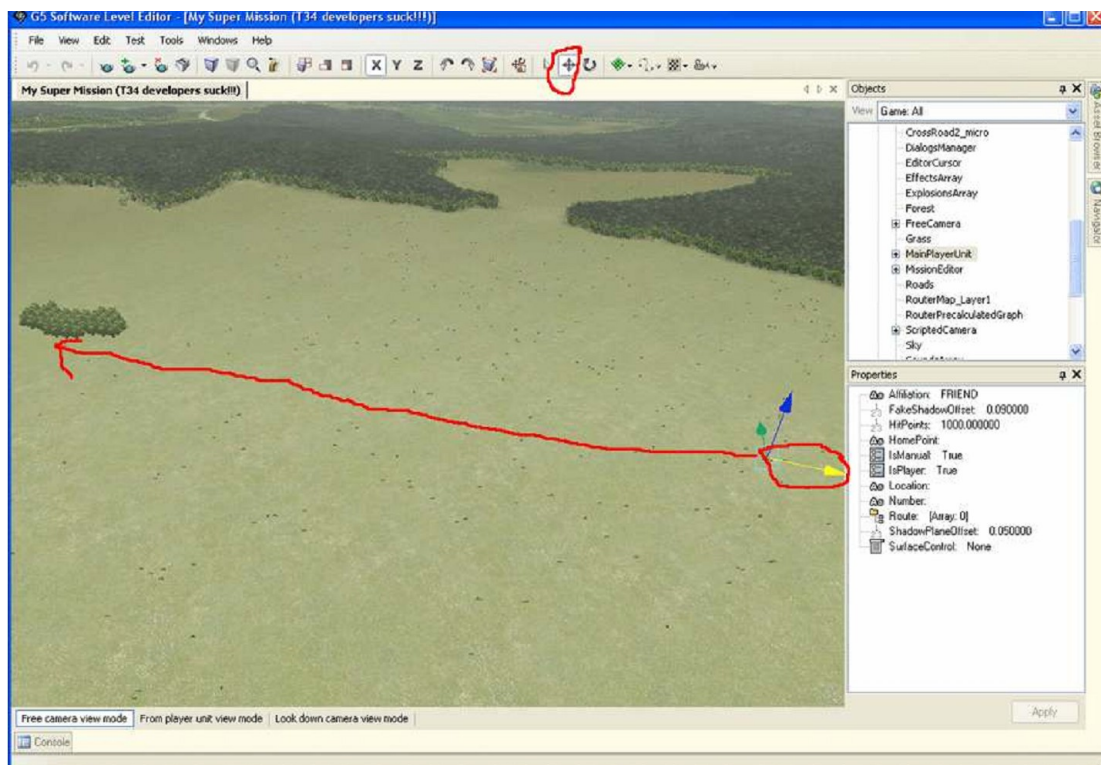
Step 5. The mission plan.

Created on our level we have one of our tank. Let's decide what will be the essence of the mission. I do not really bother with the plot, so the plan will be simple - you need to move to a nearby village, breaking the defensive line.



Step 6. Puts players in place.

Because we created a level, the player is gdeAt middle of a field rather than at the line, how would we would like, then move it back by selecting the Move tool.

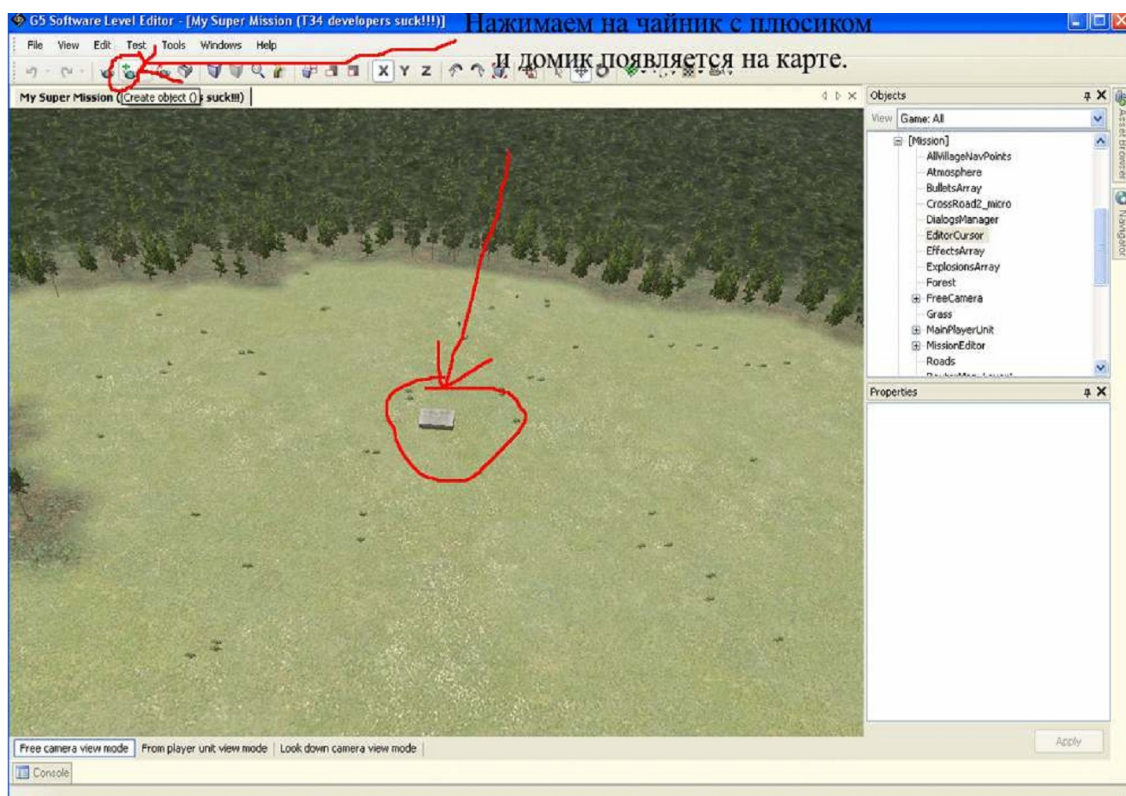
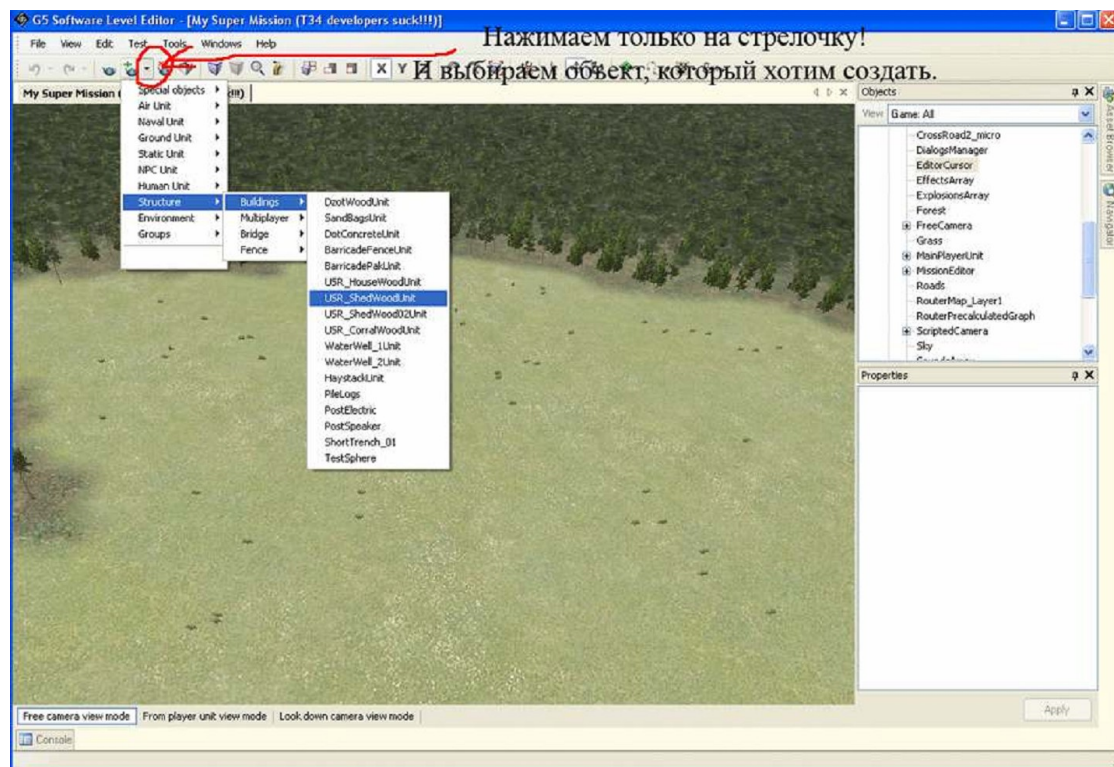


Attention! Due to the game physics, I do not recommend trying to ride atank, you moved ... Try it and see what happens ☺ Better save the level and load it again.

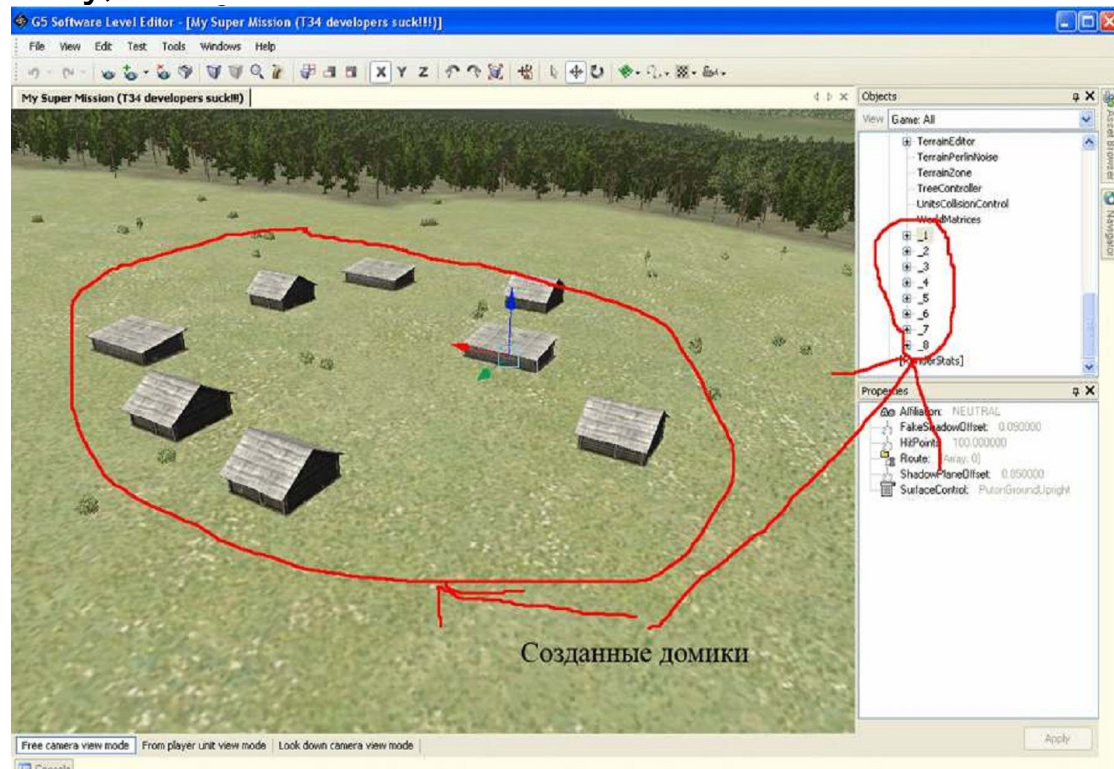
Step 7. Creation of the village.

Now create a few homes that will symbolize the captured village. Note the word "symbolize", because I do not claim to authenticity Belarusian villages.

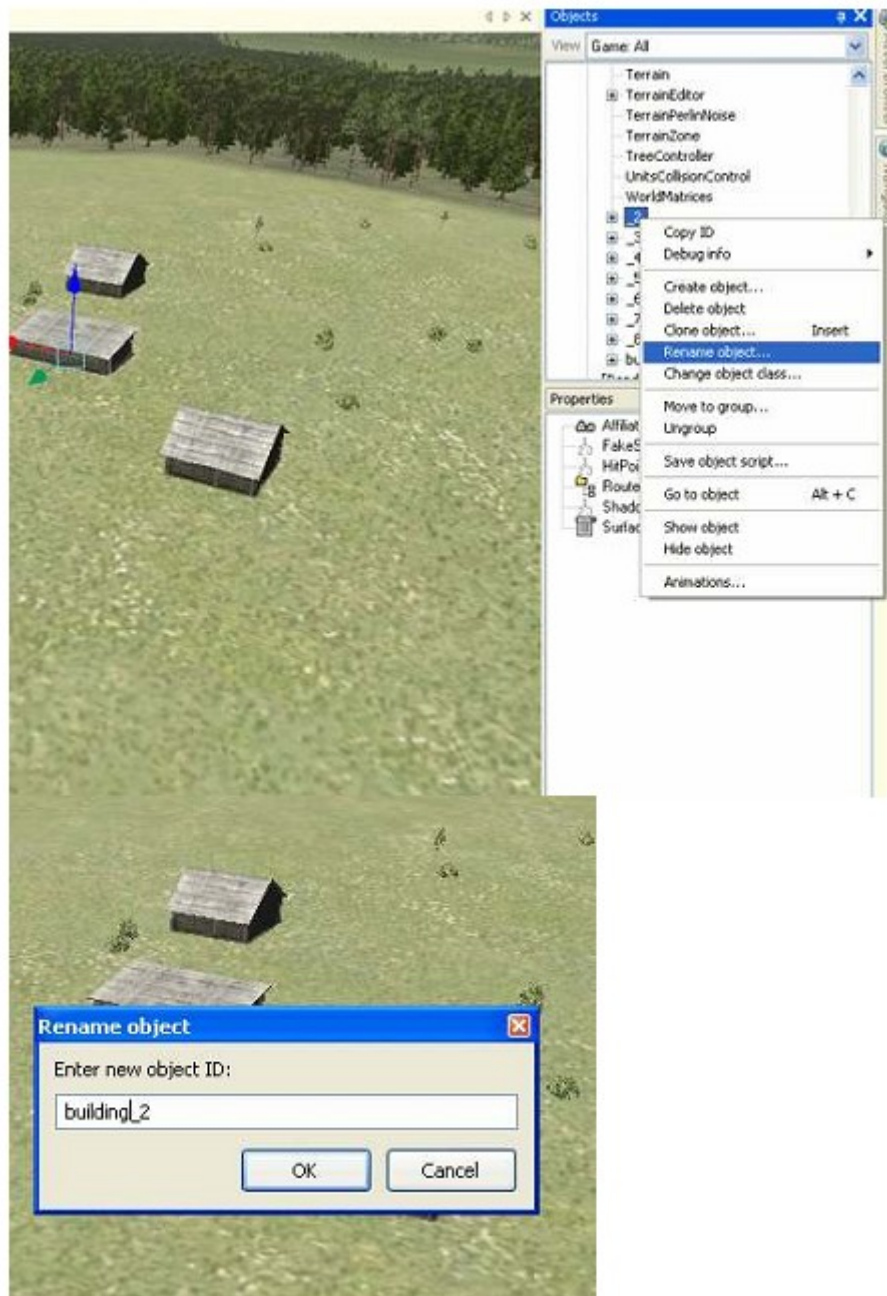
To create an object, we first need to choose its type, and then add it on the map. Screens illustrate the process of creating:



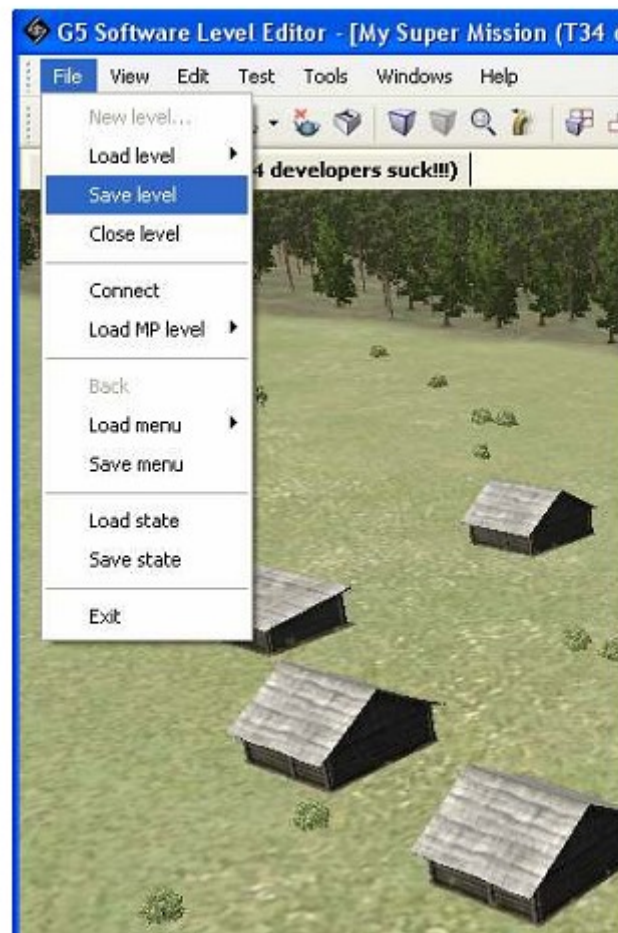
Similarly, create the rest of the house.



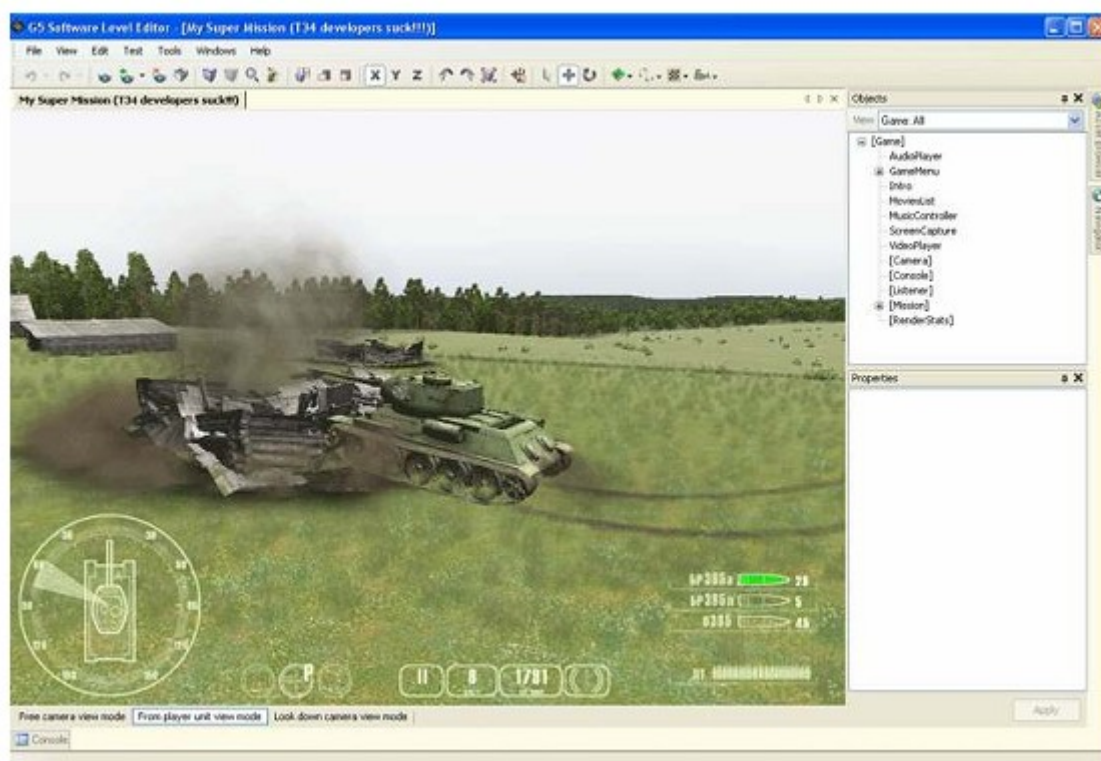
Names for objects created are automatically generated, but better them as not leave, so take care of the comfort and rename them. Right click mouse over the name of the object that we want to rename and select **"Rename object ..."**



Rename all objects that way. Once we have created a village, we keep our mission.

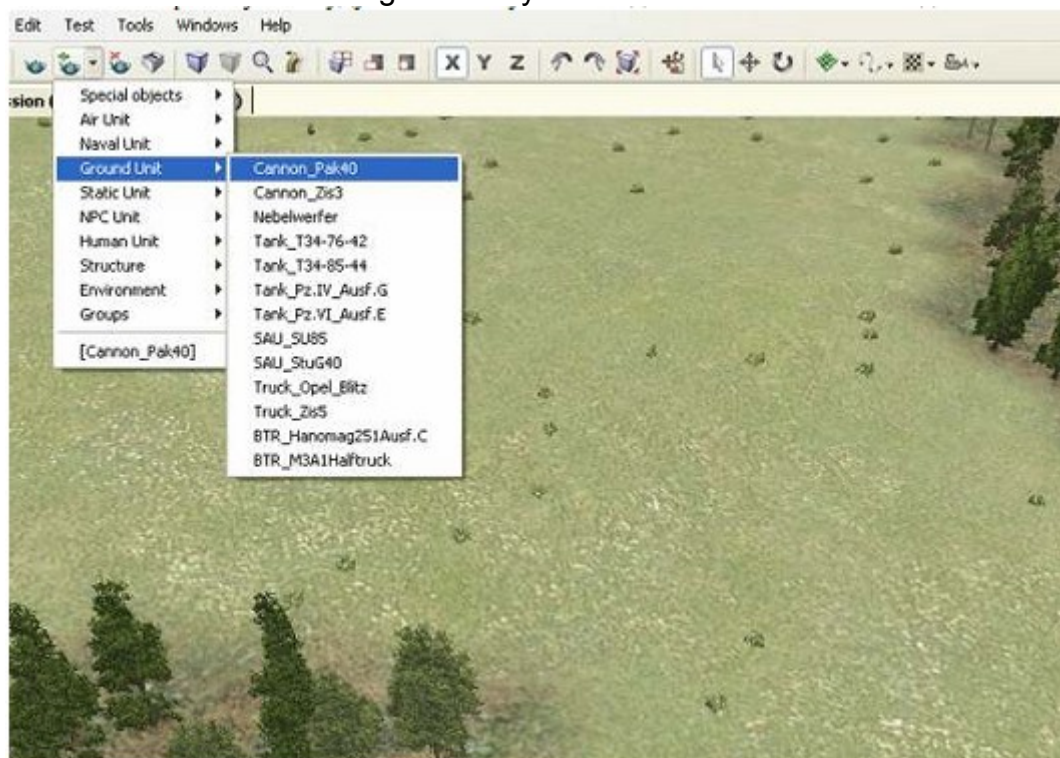


As I said earlier, all the objects that we add to the map in real time become interactive. We can drive up to our houses and destroy them.

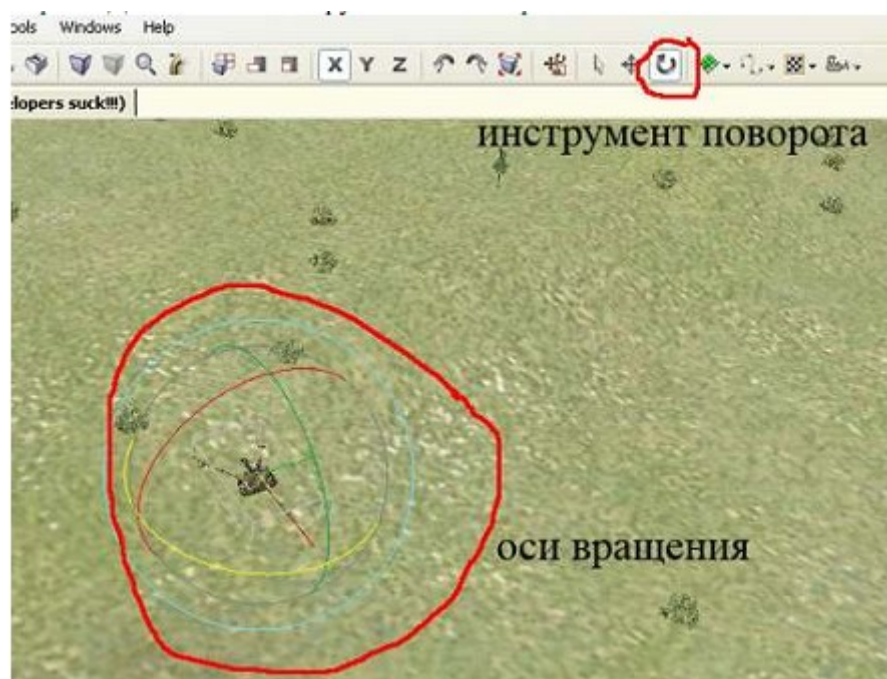


Step 8. Create a gun.

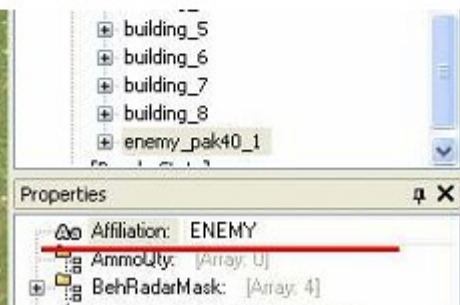
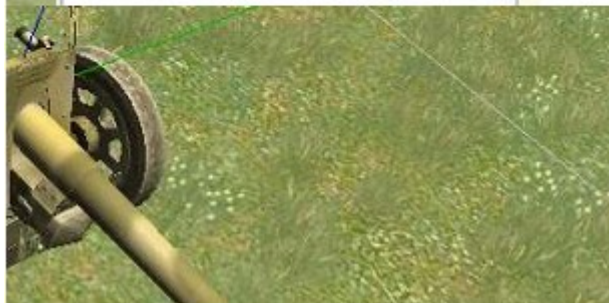
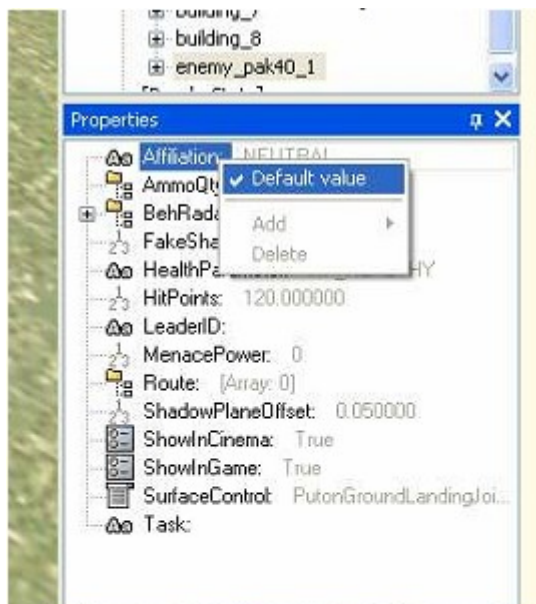
On the defensive line we will be 2 guns. They are created the same house.



Once created, you can turn it in the right direction, so that she looked at player. This is done tool "turn".



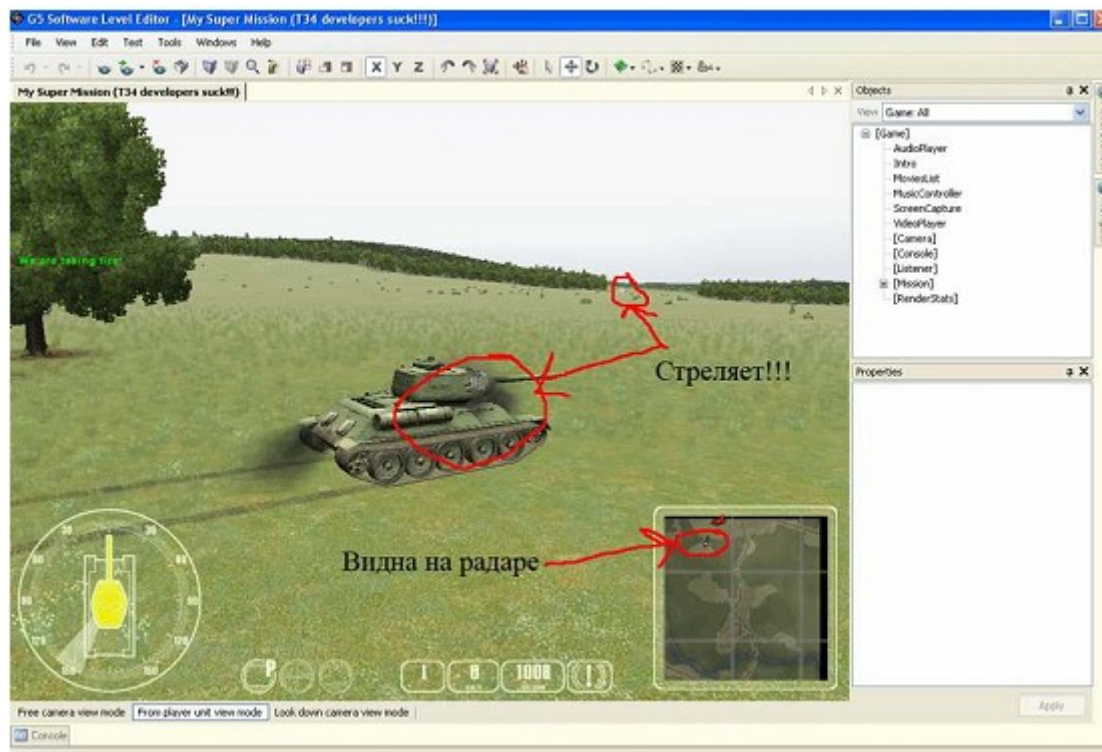
After this gun is still a static object. Need to tie it "brain" and set to "enemy" in its properties that it fired at us, and not by their own. To do this we click the right button on the property and remove the check mark Affiliation with the Default value. After which prescribes the value of «Enemy» in this field.



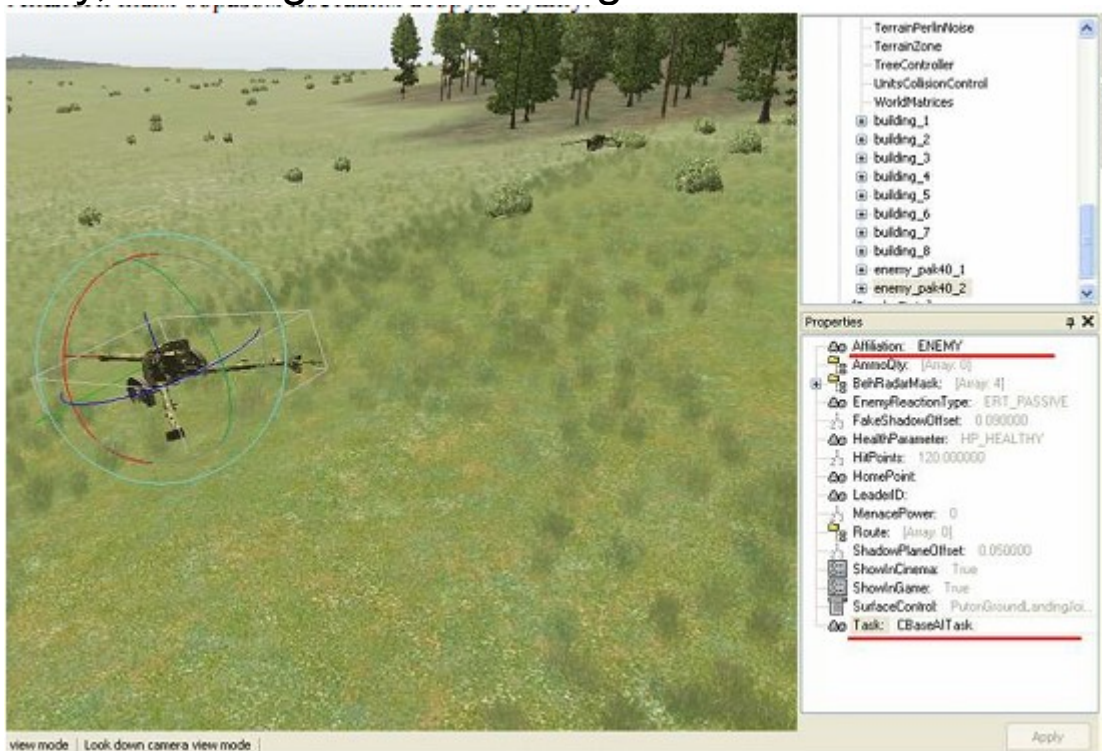
For the "brains" are responsible Tusky (Task), there is a small set of standard units for Tasco, that we can use. In this case, assign the gun «CBASEAITask».



Check how it works. Keep our level and load again. To activate the AI for the new unit, you must restart the mission. As soon as we drive out izAza woods gun will shoot at us. We have it also can destroyed.

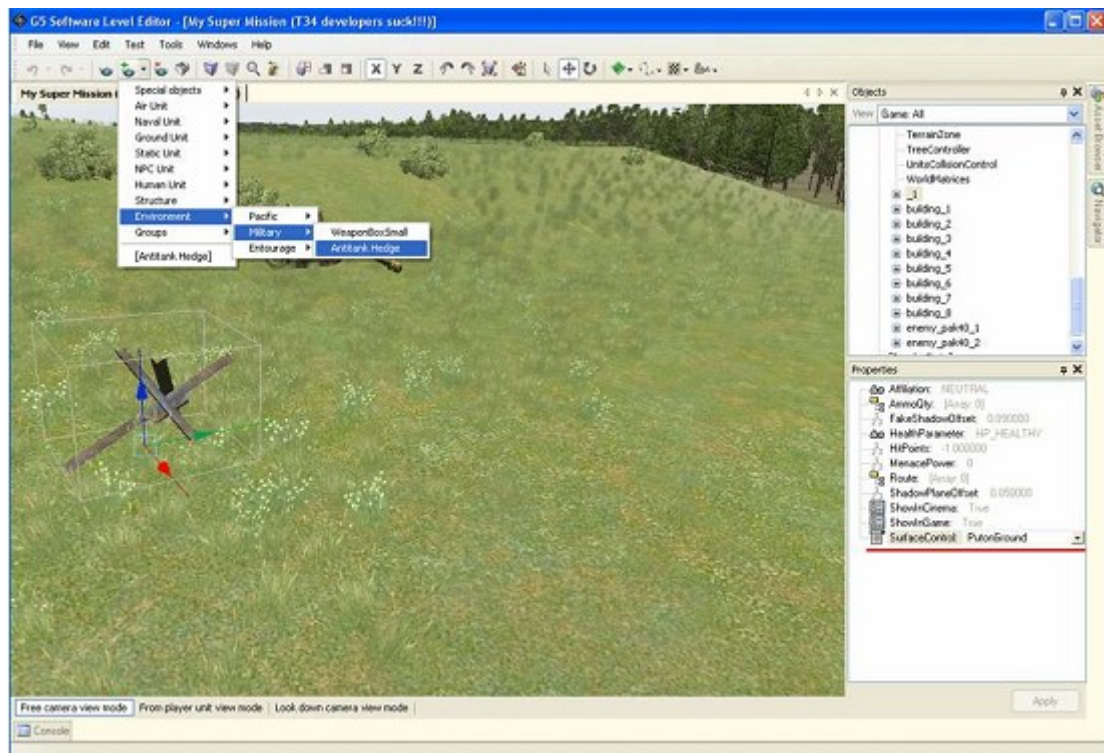


Similarly, we assign the second gun.



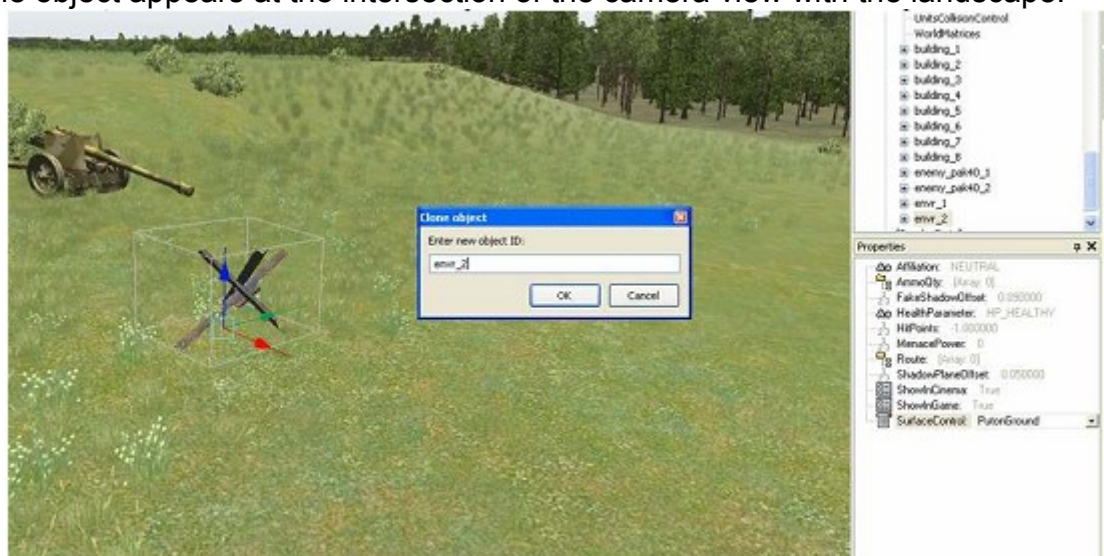
Step 9. Create the elements of the environment.

The game is good in parts! Therefore, we add elements of the environment, which diversify gaming space. Pose anti-urchins.

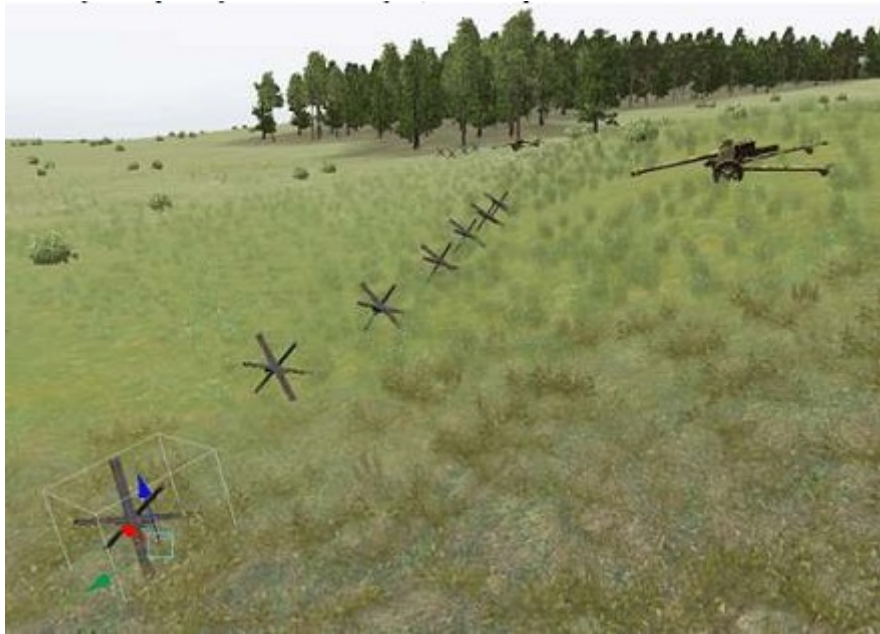


To make it convenient to move across the landscape, we can put to him property "SurfaceControl" in value "PutonGround", then it will automatically positioned on the ground.

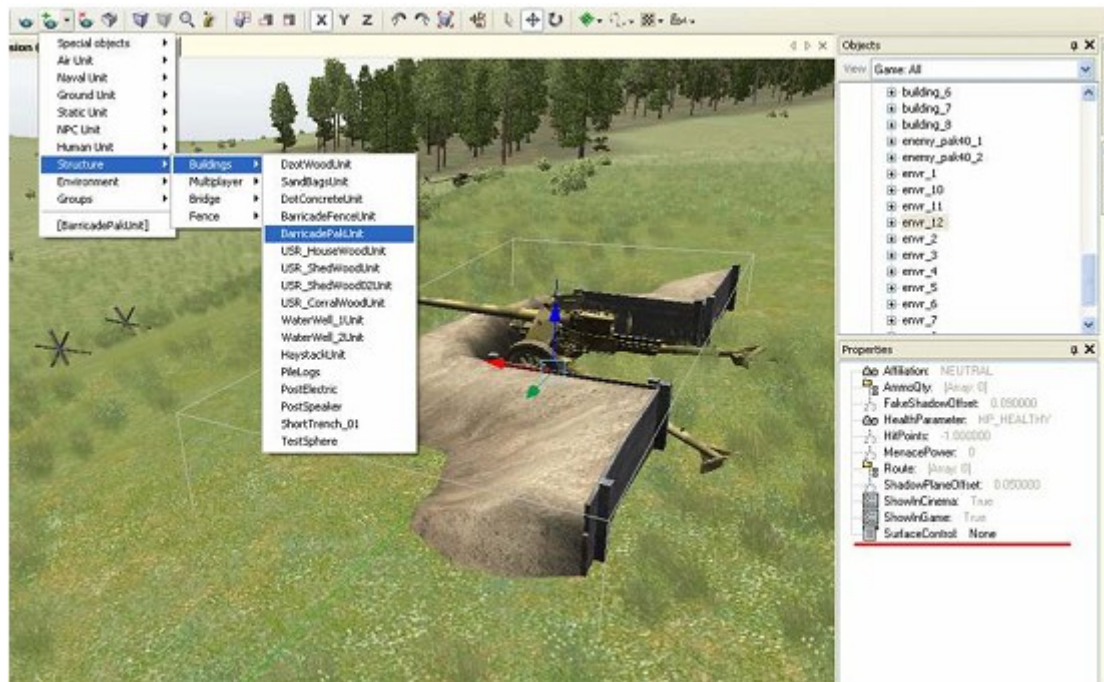
Because we want a lot of "urchins", then I'll show you a way to clone objects. For this should either right click on the name of the object and choose «Clone object», or on the game screen, click Insert, then you will be prompted to enter a name for the new object and the object appears at the intersection of the camera view with the landscape.



By repeating this operation many times, we get chtoAto like this:

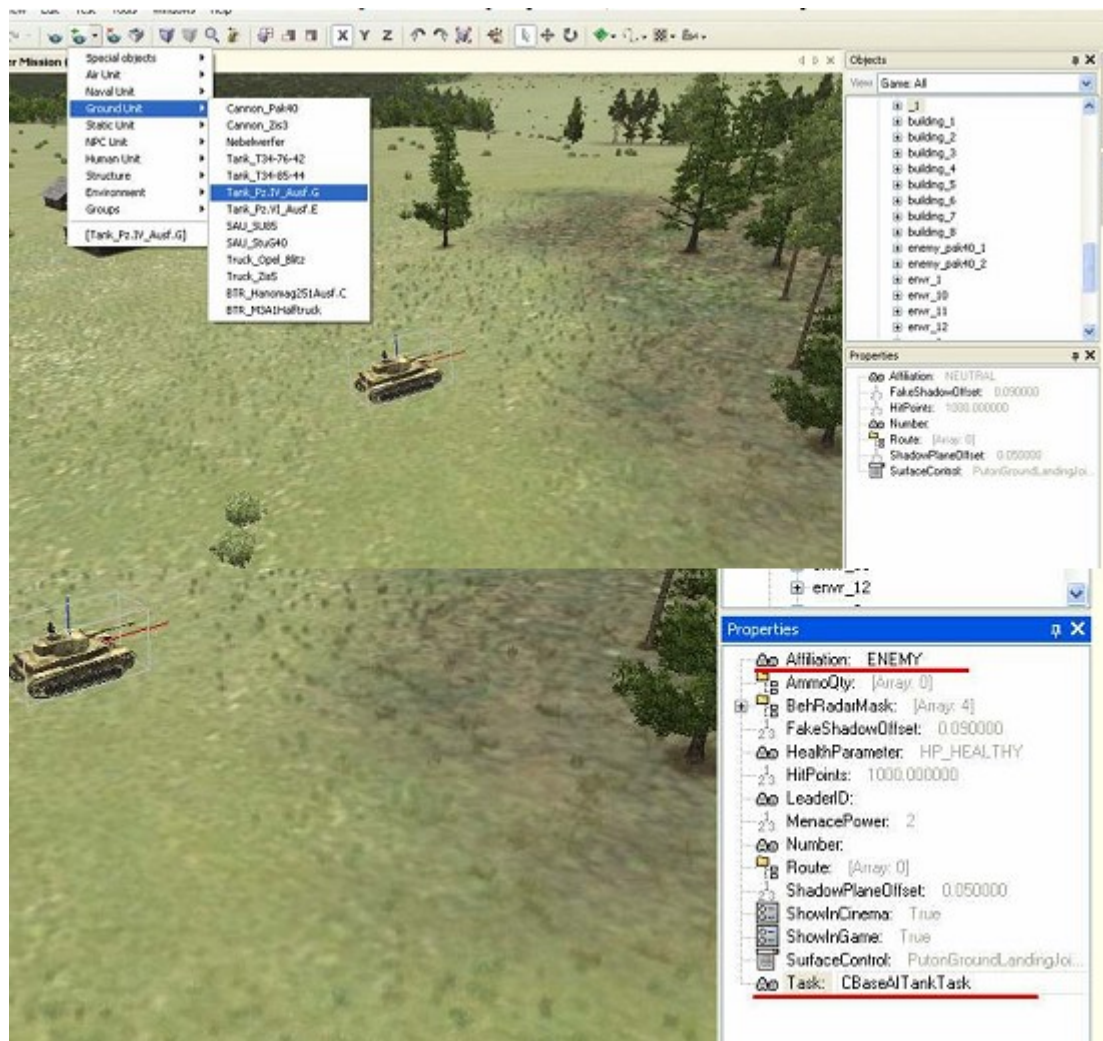


Next, close the gun will create a trench, and then assign the properties of "None" for "SurfaceControl".



Step 10. Put a tiger.

When a player overcomes the defensive line, it will wait for an enemy tank. For this we need to put it next to the village, and assign the necessary properties.

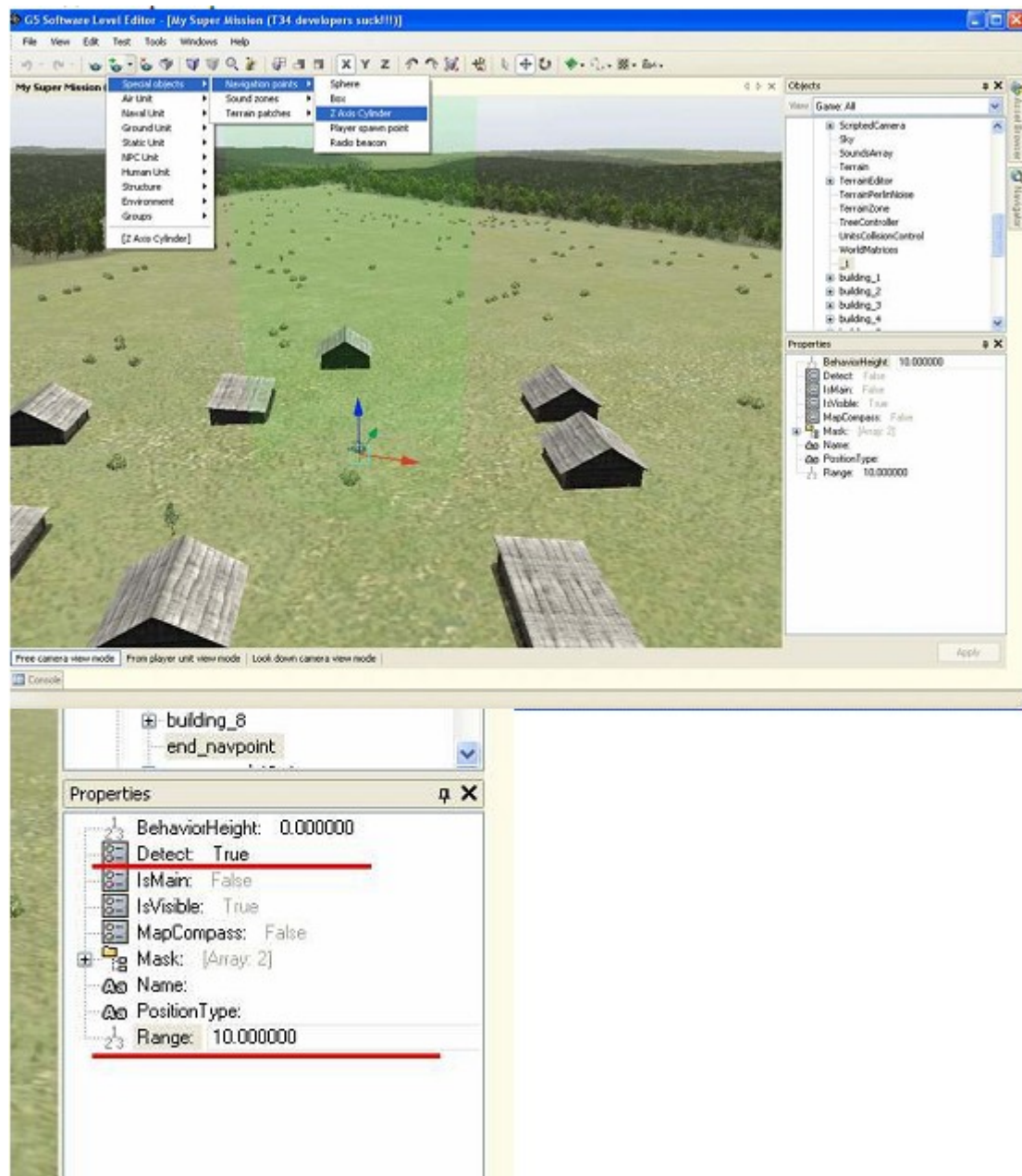


For the tanks we use CBaseAITankTask.

Do not forget to keep the level as you add new objects. You can reboot and check whether the tank is attacking you. Unfortunately, CBaseAITankTask does not include the movement from the place after the initialization is already must be set in scripts. But that's the next lesson.

Step 11. End of mission.

We want to see the mission ends when the player vedet in the village. For this we you must install the trigger and handle the event raised when an object gets into the trigger.



To trigger fires, not just standing on the ground, it is necessary to activate
 Detect = True and set the radius of him.

Handling all of the triggers are in the script of the mission. We need to check a unit came
 in and finish the mission, if it was a player. Add the following lines to the script
 Mission.script

```

void StartMission()
{
    // call inherited
    CSPMission::StartMission();

    Component console = new #GameController().GetObject(SOID_Console);
    // console.disablebeh();
    //console.showbehinf(true);
}

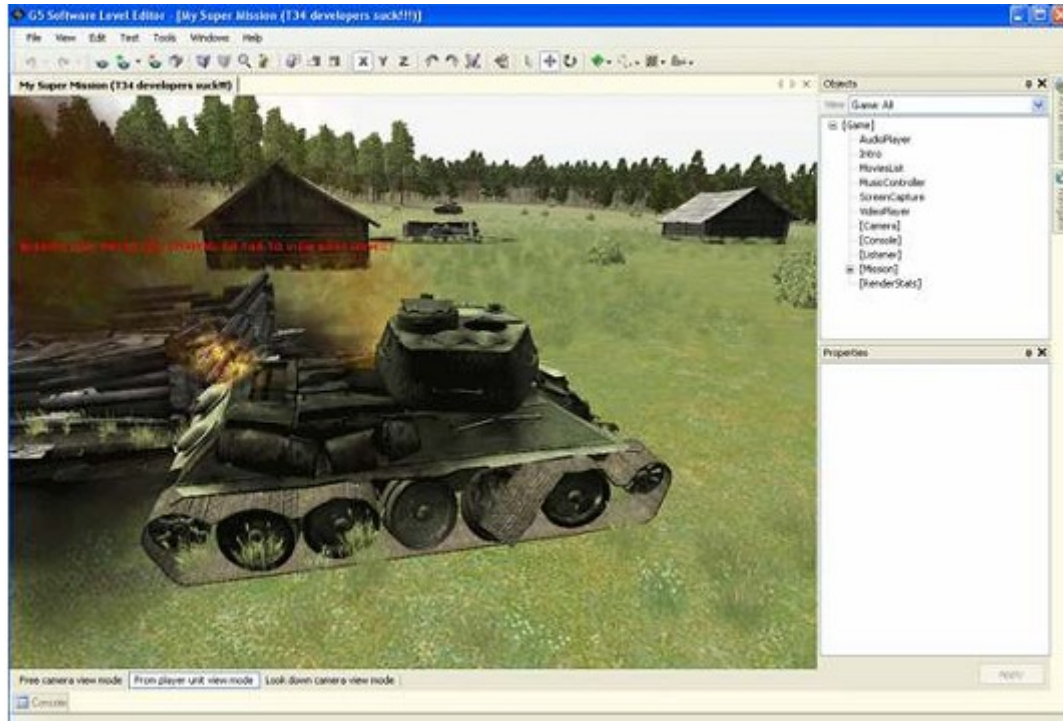
void OnObjectEnterNavPoint(String _NavPointID, String _ObjectID)
{
    if (_ObjectID == "MainPlayerUnit")
    {
        sendEvent(1.0, getIdentificator(this), "CloseMission", [MOID_Completed]);
    }
}

void OnObjectLeaveNavPoint(String _NavPointID, String _ObjectID)
{
}

void Shutdown()
{
    CSPMission::Shutdown();
}

```

Now, if we drive into the village, the Triggers and the menu will be the end game. If we blow up, you will see an appropriate caption.



End!

Translated by Google translator and edited by PzU for the T34 vs Tiger community